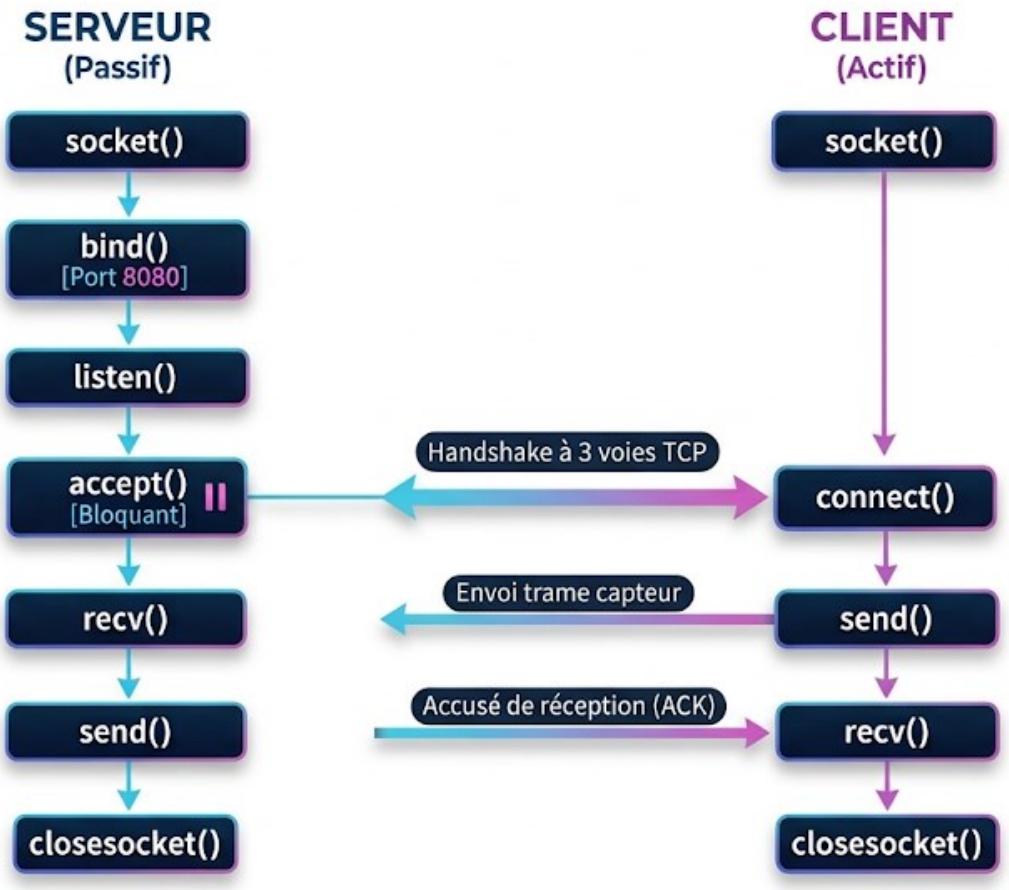
TP 1 : Communication Client-Serveur par Sockets TCP

Étape 1 : Architecture et Déroulement de la Communication

Le protocole TCP fonctionne en mode connecté (flux d'octets fiable et ordonné). L'échange repose sur un automate précis entre les fonctions de l'API Sockets :



Étape 2 : Code Source (Winsock)

1. Fichier server.c

Créez le fichier server.c dans votre répertoire de travail :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#define PORT 8080
#define BUFFER_SIZE 1024
```

```
int main() {
    WSADATA wsa;
    SOCKET server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    const char *ack_message = "ACK_SERVER: Trame recue avec succes.";

    // 1. Initialisation de la bibliothèque réseau Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("[ERREUR] Echec de l'initialisation Winsock : %d\n",
WSAGetLastError());
        return 1;
    }
    // 2. Création de la socket IPv4 (AF_INET), flux TCP (SOCK_STREAM)
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
        printf("[ERREUR] Echec creation de socket : %d\n",
WSAGetLastError());
        WSACleanup();
        return 1;
    }
    // Configuration de l'adresse d'écoute
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY; // Écoute sur toutes les interfaces
réseau
    address.sin_port = htons(PORT);        // Conversion port hôte -> ordre
réseau (Big-Endian)

    // 3. Liaison (bind) de la socket à l'adresse et au port
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) ==
SOCKET_ERROR) {
        printf("[ERREUR] Echec du bind : %d\n", WSAGetLastError());
        closesocket(server_fd);
        WSACleanup();
        return 1;
    }

    // 4. Mise en mode passif (listen) avec une file d'attente max de 3
clients
    if (listen(server_fd, 3) == SOCKET_ERROR) {
        printf("[ERREUR] Echec de listen : %d\n", WSAGetLastError());
        closesocket(server_fd);
        WSACleanup();
        return 1;
    }

    printf("========================================\n");
    printf("[SERVEUR] En attente de connexion sur le port %d...\n", PORT);
    printf("========================================\n");

    // 5. Acceptation bloquante de la connexion
```

```
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
&addrlen)) == INVALID_SOCKET) {
        printf("[ERREUR] Echec accept : %d\n", WSAGetLastError());
        closesocket(server_fd);
        WSACleanup();
        return 1;
    }
    printf("[SERVEUR] Connexion entrante acceptee !\n");

    // 6. Lecture des données envoyées par le client
    int bytes_received = recv(new_socket, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("[SERVEUR] Donnees recues : %s\n", buffer);
    }

    // 7. Envoi de l'accusé de réception au client
    send(new_socket, ack_message, (int)strlen(ack_message), 0);
    printf("[SERVEUR] Accuse de reception envoye au client.\n");

    // 8. Fermeture des sockets et libération des ressources
    closesocket(new_socket);
    closesocket(server_fd);
    WSACleanup();
    return 0;
}
```

2. Fichier client.c

Créez le fichier client.c :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    WSADATA wsa;
    SOCKET sock;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};

    // Trame simulant le capteur du TD1
    const char *sensor_payload = "ID=S01; Temp=28.5C; Humidity=63%;
Time=10:30:00";

    // 1. Initialisation Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
```

```
        printf("[ERREUR] Initialisation Winsock : %d\n", WSAGetLastError());
        return 1;
    }

    // 2. Création de la socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
        printf("[ERREUR] Creation de la socket : %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    // Configuration de la destination
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1"); // Adresse locale de
bouclage (localhost)

    // 3. Demande de connexion au serveur
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) ==
SOCKET_ERROR) {
        printf("[ERREUR] Connexion refusee. Le serveur est-il lance ?
(%d)\n", WSAGetLastError());
        closesocket(sock);
        WSACleanup();
        return 1;
    }

    printf("[CLIENT] Connecte avec succes au serveur !\n");

    // 4. Envoi de la trame
    send(sock, sensor_payload, (int)strlen(sensor_payload), 0);
    printf("[CLIENT] Trame emise : %s\n", sensor_payload);

    // 5. Réception de la réponse du serveur
    int bytes_received = recv(sock, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("[CLIENT] Reponse du serveur : %s\n", buffer);
    }

    // 6. Fermeture de la session
    closesocket(sock);
    WSACleanup();
    return 0;
}
```

Étape 3 : Compilation et Exécution dans VS Code

- Ouvrez le terminal intégré de VS Code.
- Compilez les deux programmes en ajoutant le drapeau d'édition des liens Winsock (-lws2_32) :

```
gcc server.c -o server.exe -lws2_32
gcc client.c -o client.exe -lws2_32
```

- Scindez l'écran du terminal en deux fenêtres côte à côte(bouton **Split Terminal)**
- **Dans le Terminal 1 (Serveur) :** Lancez le serveur en premier :

```
.\server.exe
```

- Le message [SERVEUR] En attente de connexion... s'affiche.
- **Dans le Terminal 2 (Client) :** Lancez le client :

```
.\client.exe
```

- Le client transmet les données, le serveur les affiche et renvoie l'accusé de réception.

Exercice Pratique

Le client doit permettre à l'utilisateur de saisir au clavier une commande ou une température (TEMP=35.0), et le serveur doit rester ouvert en boucle (while(1)) pour recevoir plusieurs messages successifs jusqu'à ce que le client envoie **QUIT**.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    WSADATA wsa;
    SOCKET server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    WSAStartup(MAKEWORD(2, 2), &wsa);

    // [A COMPLETER 1] Creer la socket TCP
    server_fd = /* ??? */;

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // [A COMPLETER 2] Lier la socket a l'IP et au port
    /* bind( ... ); */

    // [A COMPLETER 3] Mettre la socket en ecoute
    /* listen( ... ); */

    printf("[SERVEUR BOUCLE] Pret a recevoir des connexions...\n");
```

```
    // Boucle permanente du serveur pour accepter de multiples clients
successivement
    while (1) {
        printf("\n[SERVEUR] En attente d'un nouveau client...\n");

        // [A COMPLETER 4] Accepter la connexion entrante
        new_socket = /* ??? */;

        printf("[SERVEUR] Client connecte !\n");

        // Boucle de dialogue avec le client connecte
        while (1) {
            memset(buffer, 0, BUFFER_SIZE);

            // [A COMPLETER 5] Recevoir le message
            int bytes_received = /* recv( ... ); */;

            if (bytes_received <= 0) {
                printf("[SERVEUR] Client deconnecte.\n");
                break;
            }

            printf("[RECU] : %s\n", buffer);

            // Verification d'arret
            if (strncmp(buffer, "QUIT", 4) == 0) {
                const char *bye = "Session terminee.";
                send(new_socket, bye, (int)strlen(bye), 0);
                break;
            }

            // Reponse echo
            const char *ack = "Mesure enregistree.";
            send(new_socket, ack, (int)strlen(ack), 0);
        }

        // Fermeture de la socket de ce client particulier
        closesocket(new_socket);
    }

    closesocket(server_fd);
    WSACleanup();
    return 0;
}
```